提高组模拟赛题解

第 12 套

饼干（cookie）

部分分

把每盒的当前数量合在一起作为状态，BFS 枚举操作次数不超过 K 的所有状态，检查哪些状态中各盒数量相同，能得到 20 分。

仍然枚举最终数量，但按正解中的方法，只保留每层可能出现的两个数量。设 $A = \max a_i$，时间复杂度为 $O(nA \log A)$，能得到 50 分。

正解

先考虑一盒饼干。若最初有 a 块，进行 t 次操作后，数量只可能是

$$\left\lfloor \frac{a}{2^t} \right\rfloor \quad \text{或} \quad \left\lceil \frac{a}{2^t} \right\rceil .$$

可以归纳验证：将这一层的两个数分别向上、向下取半，得到的数仍在下一层的两个端点之间。连续向下取整和连续向上取整分别能得到两个端点；只保留正数，降到 1 后不再操作。这样每层至多两个数，一盒饼干能得到的不同数量只有 $O(\log A)$ 个。同一个数可能在不同层出现，只记录第一次出现的层数，就是得到它所需的最少操作次数。

用 map 为每个可达数量记录 cnt 和 sum，分别表示能得到它的盒数和这些盒子所需的最少操作次数之和。逐盒枚举，遇到一个可达数量，就将对应的 cnt 加一，并将最少操作次数累加到 sum。若 $cnt = n$ 且 $sum \le K$，这个数便可以作为答案：各盒的操作互不影响，分别按最少次数操作即可。在所有可行的数中取最大的一个。

每层先记录较大的数，再记录较小的数，得到的序列不增，跳过相邻重复值即可保证每盒对同一个数只统计一次。直接在读入后逐层枚举并更新 map，不必另外保存这些数。记 $A = \max a_i$，每盒只有 $O(\log A)$ 个可达数量，map 中至多有 $O(n \log A)$ 项，因此时间复杂度为 $O(n \log A \log(n \log A))$，空间复杂度为 $O(n \log A)$。

训练（train）

部分分

对于 $n, k, A \le 12$ 的数据，枚举每个项目练几轮，各项目的轮数都为正整数，总和为 k。若第 i 个项目练 t_i 轮，则每轮时长 d 应满足 $\lceil l_i/t_i \rceil \le d \le \lfloor r_i/t_i \rfloor$。求这些区间的交，便能得到该分配方案下最大的 d，可得 15 分。

对于 $n, A \le 2000$ 的数据，枚举 d，逐个项目计算允许的轮数范围，再按正解中的条件判断。时间复杂度为 $O(nA)$，可得 40 分。

特殊性质

当所有 $l_i = 1$ 时，每个项目至少练一轮，因此需要 $k \ge n$ 且 $d \le \min r_i$。在这个范围内，能安排的最多轮数为 $\sum_i \lfloor r_i/d \rfloor$，随 d 增大而不增，可以二分最大的可行 d。时间复杂度为 $O(n \log A)$，单独处理可得 10 分，与前两种做法分别合用可得 25 分、50 分。

正解

固定每轮时长 d。第 i 个项目练 t_i 轮，总时间就是 dt_i，因此至少要练 $\lceil l_i/d \rceil$ 轮，最多能练 $\lfloor r_i/d \rfloor$ 轮。若所需的最少轮数比允许的最多轮数还大，这个项目便无法安排。

每个项目都能安排后，把最少要练的轮数相加，若超过 k，就连最低要求也无法满足；把最多能练的轮数相加，若不足 k，就凑不够。否则，先给每个项目安排最少的轮数，再给还能继续练的项目逐轮增加，就能恰好凑到 k 轮。

例如样例第一组，$d = 4$ 时，两个项目分别可以练 $1 \sim 2$ 轮、1 轮，总共可以安排 $2 \sim 3$ 轮。要求 $k = 3$，就在各练一轮后，给第一个项目再加一轮。

先检查每个项目能否安排，即 $[l_i, r_i]$ 中是否有 d 的倍数。如下图，$d = 4$ 时，要求练 $5 \sim 7$ 分钟的项目无法安排：一轮只有 4 分钟，两轮又有 8 分钟。

AUDITFIG0

不含 d 的倍数，就是存在 $q \ge 0$，使 $qd < l_i \le r_i < (q+1)d$。预处理 $f[x]$，表示所有 $r_i \le x$ 的项目中最大的 l_i，没有则取 0。按 r_i 记录最大的 l_i，再求前缀最大值即可。

枚举相邻倍数 $qd, (q+1)d$，令 $x = \min(A, (q+1)d - 1)$。若 $f[x] > qd$，就有一个项目的 $l_i > qd$ 且 $r_i \le x$，它要求的时间全在两个倍数之间，这个 d 便不行。检查时也要包含 A 以内最后不足 d 的一段。

再统计总轮数。令 $L[x]$ 为 $l_i \ge x$ 的项目数，$R[x]$ 为 $r_i \ge x$ 的项目数，分别统计 l_i, r_i 的出现次数，再求后缀和即可。每个项目都需要第一轮；$l_i > d$ 的还需要第二轮，$l_i > 2d$ 的还需要第三轮。类似地，允许练第一轮的项目有 $R[d]$ 个，允许练第二轮的有 $R[2d]$ 个。逐轮统计，得到

$$\begin{aligned} &\text{最少总轮数} = n + L[d+1] + L[2d+1] + L[3d+1] + \cdots, \\ &\text{最多总轮数} = R[d] + R[2d] + R[3d] + \cdots. \end{aligned}$$

一次判断只需 $O(A/d + 1)$ 时间。可行性不单调，不能直接二分；从 A 到 1 枚举 d，第一个可行值就是答案，都不行则输出 -1。总时间复杂度为 $O(n + A \log A)$，空间复杂度为 $O(A)$。

合影（photo）

部分分

每次变动后，逐个社团统计到场人数。一个社团有 c 人到场，就贡献 $c(c-1)/2$ 张合影。时间复杂度为 $O(qM)$，可得 15 分。

特殊性质

只需修改发生变动的成员所在社团。若某社团原有 c 人到场，新增一人时答案增加 c，离开一人时答案减少 $c-1$。性质 A 下每次至多修改 20 个社团；性质 B 下至多修改一个社团。单独处理性质 B 可得 5 分，结合小数据可得 20 分；处理性质 A 也能通过性质 B，结合小数据共可得 35 分。

正解

记 G_u 为成员 u 参加的社团集合，$d_u = |G_u|$。一次变动对答案的影响，是 u 与其他到场成员共同参加的社团数之和。直接遍历 G_u 在 d_u 较大时仍然太慢。

取 $B = \lfloor\sqrt{M}\rfloor$，将 $d_u > B$ 的成员称为重成员，其余称为轻成员。因为 $\sum_u d_u = M$，重成员不超过 M/B 个。对于每名成员 u 和每名重成员 h，预处理 $f_{u,h} = |G_u \cap G_h|$：标记 h 所在的社团，再遍历所有成员的名单，统计命中标记的次数。共同参加几个社团就计几次，不能只判断是否有交集。

将到场成员的贡献分开维护。cnt_i 只记录社团 i 中到场的轻成员数；对每名重成员 h，维护 $sum_h = \sum_{i \in G_h} cnt_i$。查询 u 的贡献时，若 u 是轻成员，直接求 $\sum_{i \in G_u} cnt_i$；否则取 sum_u。再枚举所有已经到场的重成员 h，加上 $f_{u,h}$，记结果为 res。

AUDITFIG1

图中 $M = 15, B = 3$，重成员为 $5, 6$。成员 1 到场时，与轻成员 $2, 3$ 各增加一张，与重成员 $5, 6$ 分别增加两张、一张，共增加 5 张。

若 u 到场，答案增加 res；若 u 离开，查询时她仍在场，res 将她自己在每个所属社团中各计了一次，所以答案减少 $res - d_u$。之后修改到场状态。轻成员 u 到场或离开时，将所有 $i \in G_u$ 的 cnt_i 加一或减一，同时将每个 sum_h 加上或减去 $f_{u,h}$；重成员变动时只需修改到场状态，后续查询枚举重成员时自然会得到新的贡献。

上述维护始终分别统计到场轻成员和重成员，恰好包含所有应增加或减少的合影；离开时扣去自身的计数，因此每次更新后的答案正确。

预处理的时间复杂度为 $O((n+M)M/B)$，每次变动为 $O(B + M/B)$。取上述 B 后，总时间复杂度为 $O((n+M+q)\sqrt{M})$，空间复杂度为 $O(n\sqrt{M} + n + m + M)$。

路线（route）

部分分

$n, m \leq 150$ 时，反复扫描所有路线，遇到当前边权小于前一条边时就将它提高。每轮能将一个初始边权沿比较关系多传递一步，至多 $n-1$ 轮后稳定。时间复杂度 $O(n^2 m)$，可得 15 分。

$n, m \leq 3000$ 时，枚举每条路线上的相邻两条边，用 $O(nm)$ 时间建立正解中的有向图，再求各边必须达到的最小值，可得 35 分。

特殊性质

树是一条链时，按链上的顺序给边编号。一条路线要求一个区间内的相邻边依次不下降，或依次不增加。分别对两个方向作区间差分，便能判断每对相邻边之间有哪些比较关系，再建图求解。时间复杂度为 $O(n \log n + m)$，这一性质占 15 分；与前两种做法结合，分别可得 30 分、50 分。

正解

记每条小路的最终难度为 b_e。将原树的每条边作为有向图中的一个点；若某条路线先走过 e，紧接着走过 f，就连 $e \to f$，表示 $b_e \leq b_f$。这些关系还会在不同路线之间传递。

若 e 能到达 f，就有 $b_f \geq b_e \geq a_e$。因此，b_f 应取所有能到达它的点的初始难度最大值，包括它自己。这样不会违背大小关系：对于 $e \to f$，能到达 e 的点也都能到达 f，所以 $b_e \leq b_f$。各边都已取到最小值，答案为 $\sum_e (b_e - a_e)$。

按初始难度从大到小枚举各点，从未赋值的点出发 DFS，把沿途未赋值的点都赋为本次的初始难度。已访问点及其后续能到达的点都由此前的搜索处理，不必重复访问。一个点首次被赋值时，若还有更大的初始难度能传到它，它就应当更早被访问，所以此时的值正是所求的最大值。

用边差分加快建图：以 1 为根，记 x 的父亲为 fa_x、父边为 e_x，用非根节点 x 代表图中对应 e_x 的点。设 $w = \mathrm{LCA}(s, t)$，先考虑 s, t 都不等于 w，记 p, q 为 w 通向 s, t 的儿子。

AUDITFIG2

图中要连 $s \to p$、$p \to q$、$q \to t$。转弯处的 $p \to q$ 单独加入，其余比较都发生在祖先链上，用边差分处理。

上行时，对路径 $s \to p$ 上的每条边 (fa_x, x)，需要连 $x \to fa_x$。令 up_s 加一、up_p 减一，再求子树和，$up_x > 0$ 就连这条有向边。加一传到 p 时恰好抵消，不会多连 $p \to w$，因为路线不经过 w 的父边。

下行同理，在路径 $q \to t$ 上令 $down_t$ 加一、$down_q$ 减一，求子树和后，$down_x > 0$ 就连 $fa_x \to x$。图中这一条路线会使 $up_s = down_t = 1$，得到两侧的 $s \to p$、$q \to t$。若 $s = w$ 或 $t = w$，跳过对应一侧；两侧都存在时才连 $p \to q$。

所有路线的标记一起累加，最后求一次子树和。这些关系共 $O(n + m)$ 个，也不一定要全部存进邻接表：DFS 到 x 时，按 $up_x > 0$ 访问父亲，按 $down_y > 0$ 访问儿子 y，只另外保存 $p \to q$。配合倍增求 LCA 和 p, q，总时间复杂度为 $O((n+m)\log n)$，空间复杂度为 $O(n \log n + m)$。